

Predicción de errores en Ingeniería de Software

Preprocesamiento

Raúl Cruz Barbosa

Instituto de Computación

Universidad Tecnológica de la Mixteca



Outline

- 1 Preprocesamiento
 - Introducción
 - Limpieza de Datos
 - Transformación de datos
 - Balanceo de clases
 - Agrupamiento
 - Introducción
 - Agrupamiento k-Means
 - Agrupamiento k-Medians
 - Agrupamiento k-Medoids
 - Extracción de características con PCA

Outline

- 1 Preprocesamiento
 - **Introducción**
 - Limpieza de Datos
 - Transformación de datos
 - Balanceo de clases
 - Agrupamiento
 - Introducción
 - Agrupamiento k-Means
 - Agrupamiento k-Medians
 - Agrupamiento k-Medoids
 - Extracción de características con PCA

Introducción

Antes de comenzar a preprocesar los datos es necesario conocer algunos tópicos inherentes a los datos, como son:

- **Formatos de datos** (archivo) estándar: CSV, XML, SQL, JSON. La mejor representación depende del consumidor final.

Ejercicio: Obtener o descargar un archivo con cada uno de los formatos mencionados arriba. Después, abrir el archivo y mostrar su contenido.

Sugerencia: utilice la biblioteca PANDAS para esta práctica.

Práctica

Instalar las siguientes de bibliotecas de Python esenciales para Ciencia de Datos:

- Verificar que el sistema tiene instalado una versión de Python3 (\$ python3 -V)
- Numpy
- Pandas
- Matplotlib
- Scikit-learn
- Jupyter Notebook environment

Introducción

- **Coleccionar o recopilar los datos.** Encontrar el conjunto de datos correcto es un tópico crucial.
 - ¿Quién podría tener los datos que necesito?
 - ¿Porqué podrían decidir proporcionarmelos?
 - ¿Cómo puedo practicar con ellos?

Algunos conjuntos de datos públicos para practicar distintas tareas se encuentran disponibles en

- UCI repository
- Kaggle datasets
- R-bloggers datasets

Introducción

Ejercicio: Accesar a las sitios mencionados anteriormente y descargar dos conjuntos de datos (por sitio) relacionados con **sistemas de recomendación**.

Calidad de Datos: ¿Por qué Pre-procesar los Datos?

Los datos tienen **calidad** si estos satisfacen los requerimientos del uso intencionado.

Existen muchos factores que involucran la calidad de datos, incluyendo *exactitud*, *completez*, *consistencia*, *oportunidad*, *credibilidad* e *interpretabilidad*.

Datos inexactos, incompletos e inconsistentes son propiedades comunes de bases de datos y data-warehouses grandes en el mundo real.

Tareas Principales en el Pre-procesamiento de Datos

Tareas principales: *limpieza de datos*, *integración de datos*, *reducción de datos*, y *transformación de datos*.

Las rutinas de **limpieza de datos** trabajan para “limpiar” los datos mediante el relleno/completado de datos faltantes, suavizado de datos con ruido, identificando o removiendo datos atípicos y resolviendo inconsistencias.

La **integración de datos** ayuda a integrar múltiples bases de datos, archivos, etc. Esto es, incluir datos de múltiples fuentes.

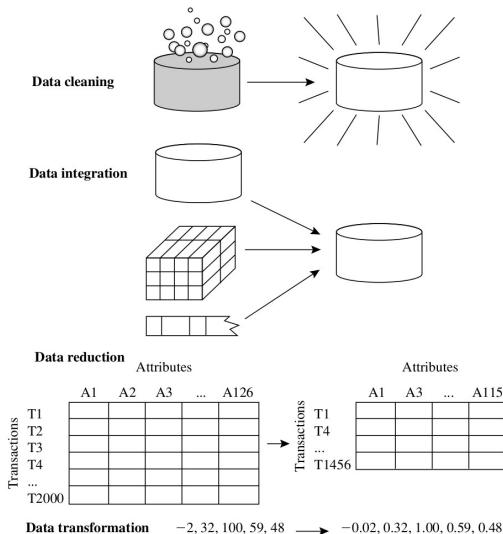
La **reducción de datos** obtiene una representación reducida del conjunto de datos que es mucho más pequeña en volumen y produce el mismo (o casi el mismo) resultado analítico. Las estrategias de reducción de datos incluyen a la reducción de dimensionalidad y reducción de muestras.

Tareas Principales en el Pre-procesamiento de Datos

Las técnicas de **transformación de datos** incluyen: *normalización*, *discretización de datos* y *generación de jerarquía de conceptos*.

Estas operaciones se pueden ver como procedimientos de pre-procesamiento adicionales que contribuirán hacia el éxito del proceso de minería/clasificación.

Tareas Principales en el Pre-procesamiento de Datos



Outline

- 1 Preprocesamiento
 - Introducción
 - **Limpieza de Datos**
 - Transformación de datos
 - Balanceo de clases
 - Agrupamiento
 - Introducción
 - Agrupamiento k-Means
 - Agrupamiento k-Medians
 - Agrupamiento k-Medoids
 - Extracción de características con PCA

Métodos Básicos de Limpieza de Datos

Datos faltantes (Missing values)

Aquí, algunas tuplas (patrones/ejemplos) no tienen un valor registrado para uno o varios atributos (variables). Técnicas para rellenar los datos faltantes de atributos.

- **Ignorar la tupla.** Esto se hace generalmente cuando se carece de la etiqueta de clase. Este método no es muy efectivo, a menos que la tupla contenga muchos atributos con valores faltantes.
- **Rellenar los valores faltantes manualmente.** Este enfoque consume mucho tiempo y puede no ser factible dado un dataset grande con muchos valores faltantes.
- **Usar una constante global para rellenar el valor faltante.** Reemplazar todos los valores faltantes de atributos por la misma constante tal como una etiqueta llamada “Desconocido” o ∞ .

Métodos Básicos de Limpieza de Datos

- **Usar una medida de tendencia central del atributo para rellenar en el valor faltante.** La media (promedio) o mediana son utilizados dependiendo de la suposición de la distribución de datos.

patrón	C_1	$C_2 \dots$	C_d
x_1	x_{11}	$x_{12} \dots$	x_{1d}
x_2	x_{21}	? $\dots$	x_{2d}
$\vdots$	$\vdots$	$\vdots \dots$	$\vdots$
x_N	x_{N1}	$x_{N2} \dots$	?

- **Usar la media o mediana del atributo para todas las muestras que pertenecen a la misma clase como la tupla dada.**

Métodos Básicos de Limpieza de Datos

patrón	C_1	$C_2 \dots$	C_d	target/salida
x_1	x_{11}	$x_{12} \dots$	x_{1d}	1
x_2	x_{21}	? ...	x_{2d}	2
$\vdots$	$\vdots$	$\vdots \dots$	$\vdots$	$\vdots$
x_N	x_{N1}	$x_{N2} \dots$	x_{Nd}	2

- **Usar el valor más probable para rellenar el valor faltante.** Esto puede ser determinado con herramientas basada en inferencia y regresión utilizando un formalismo Bayesiano, o inducción de árbol de decisión. (Utilizando los otros atributos del dataset se podría predecir el valor de un atributo)

Práctica

Utilizando una herramienta o lenguaje de programación, realice lo siguiente:

- 1 Descargar un conjunto de datos que contenga datos faltantes del repositorio UCI.
- 2 Identificar los datos faltantes y aplicar las técnicas descritas anteriormente para rellenar la matriz de datos.
- 3 Elaborar un reporte en una libreta de Jupyter Notebook.

Limpieza de datos

Eliminación de Datos Atípicos (Outliers)

Un dato atípico (outlier) es un punto que está muy lejos de la media de la variable aleatoria correspondiente. Esta distancia se mide con respecto a un umbral dado (generalmente un número de veces la desviación estandard).

Para una variable aleatoria distribuida normalmente (Gaussiana), una distancia de dos veces la desviación estandard cubre 95 % de los puntos, y una distancia de 3 veces la desv. estandard cubre 99 % de los puntos.

Los puntos con valores muy diferentes del valor medio producen errores grandes durante el entrenamiento y pueden tener efectos desastrosos.

Los efectos son aún peores cuando los outliers son el resultado de mediciones ruidosas.

Práctica: Eliminación de Outliers

- De los 3 conjuntos obtenidos en la práctica anterior, encontrar/detectar los outliers de sus conjuntos de datos, en caso de haber, y eliminarlos de dicho conjunto.
- La fórmula utilizada en problemas estadísticos es la siguiente:

$$abs(X - mean(X)) \geq n * std(X)$$

- Elaborar un reporte en una libreta de Jupyter Notebook.

Tarea: Investigar la técnica de la Desviación Absoluta de la Mediana (MAD) y realizar un reporte en una libreta de Jupyter Notebook.

Fecha: siguiente clase.

Outline

- 1 Preprocesamiento
 - Introducción
 - Limpieza de Datos
 - **Transformación de datos**
 - Balanceo de clases
 - Agrupamiento
 - Introducción
 - Agrupamiento k-Means
 - Agrupamiento k-Medians
 - Agrupamiento k-Medoids
 - Extracción de características con PCA

Feature Normalization

In general, normalization is any process that makes something more normal (it means conforming to some regularity or rule).

Usually, it means to map entries of a data set from their natural range to values between 0 and 1 (or $[-1, 1]$).

Normalization is a process of scaling the numbers in a data set to improve the accuracy of the subsequent numeric computations.

Normalization is also known as a **standardizing** process in the machine learning area.

Feature Normalization techniques

When using a learning method that treats numeric attributes or variables a normalization process could be required.

Normalization or standardizing the data is reasonable when the variables are in different units or when the variance of the different columns (variables) is substantial.

Attributes are often normalized to lie in a fixed range (e.g. from zero to one) by assuming that $\mathbf{x}$ is a numeric variable or attribute with n observed values $x_1, x_2, \dots, x_n$, then:

- dividing all values by the maximum value encountered in the attribute:

$$x'_i = x_i / \max_{\mathbf{x}}$$

Feature Normalization techniques

- subtracting the minimum value and dividing by the range between the maximum and the minimum values:

$$x'_i = (x_i - \min_{\mathbf{x}}) / (\max_{\mathbf{x}} - \min_{\mathbf{x}});$$

- calculating the statistical mean and standard deviation of the attribute values, subtract the mean from each value, and divide the result by the standard deviation. As a result the new values of the statistical variable have zero mean and unit standard deviation:

$$x'_i = (x_i - \bar{\mathbf{x}}) / \sigma_{\mathbf{x}};$$

- dividing each column by its standard deviation:

$$x'_i = x_i / \sigma_{\mathbf{x}}.$$

Práctica

Utilizando una herramienta o lenguaje de programación, realice lo siguiente:

- 1 Descargar un conjunto de datos del repositorio UCI.
- 2 Normalizar los datos aplicando las técnicas descritas anteriormente.
- 3 Elaborar un reporte en una libreta de Jupyter Notebook.

Outline

1 Preprocesamiento

- Introducción
- Limpieza de Datos
- Transformación de datos
- **Balanceo de clases**
- Agrupamiento
 - Introducción
 - Agrupamiento k-Means
 - Agrupamiento k-Medians
 - Agrupamiento k-Medoids
- Extracción de características con PCA

Introducción

La distribución de clases en muchas aplicaciones no está balanceada. Ej: actividad de tarjetas de crédito: normal o fraudulenta.

La mayoría de veces la actividad de una tarjeta de crédito sería normal y un porcentaje muy pequeño como fraudulenta.

A la clase que predomina en número de ejemplos se le llama **clase mayoritaria** y a la que contiene pocos ejemplos, **clase minoritaria**.

Para solucionar lo anterior, existen dos formas: a) reponderación de ejemplos, y b) remuestreo de ejemplos, siendo este último el más utilizado.

Métodos de muestreo

El muestreo puede ser ejecutado con o sin reemplazo. La clase rara puede ser **sobre-muestreada**, o la clase normal puede ser **sub-muestreada**, o ambas.

El sobremuestreo sencillo consiste en duplicar los ejemplos de la clase minoritaria.

El submuestreo sencillo elimina muestras de la clase mayoritaria.

La idea de ambos métodos es que ambas clases se encuentren balanceadas proporcionalmente antes de comenzar el entrenamiento de un algoritmo de clasificación.

Métodos de muestreo

Sobremuestreo sintético: SMOTE

Para evitar el sobremuestreo de muestras repetidas (provocando sobre-ajuste) como lo hace el método sencillo, el sobremuestreo sintético es propuesto.

El enfoque SMOTE funciona como sigue:

- Para cada ejemplo minoritario, determinar sus k -vecinos más cercanos pertenecientes a la misma clase.
- Dependiendo del nivel de sobremuestreo requerido, se selecciona una fracción de ellos aleatoriamente.
- Para cada par de ejemplo-vecino, se genera un ejemplo sintético sobre el segmento de línea que conecta al ejemplo minoritario a su vecino más cercano.
- La posición exacta del ejemplo es seleccionada uniformemente de manera aleatoria a lo largo del segmento de línea.

Métodos de muestreo

Los nuevos ejemplos minoritarios son agregados a los datos de entrenamiento, y el clasificador es entrenado con los datos aumentados.

Métodos de muestreo

Algoritmo original

Algorithm *SMOTE*(T , N , k)

Input: Number of minority class samples T ; Amount of SMOTE $N\%$; Number of nearest neighbors k

Output: $(N/100) * T$ synthetic minority class samples

1. *(* If N is less than 100%, randomize the minority class samples as only a random percent of them will be SMOTEd. *)*
2. **if** $N < 100$
3. **then** Randomize the T minority class samples
4. $T = (N/100) * T$
5. $N = 100$
6. **endif**
7. $N = (int)(N/100)$ *(* The amount of SMOTE is assumed to be in integral multiples of 100. *)*
8. k = Number of nearest neighbors
9. $numattrs$ = Number of attributes
10. $Sample[][]$: array for original minority class samples
11. $newindex$: keeps a count of number of synthetic samples generated, initialized to 0
12. $Synthetic[][]$: array for synthetic samples
(Compute k nearest neighbors for each minority class sample only. *)*
13. **for** $i \leftarrow 1$ **to** T
14. Compute k nearest neighbors for i , and save the indices in the $nnarray$
15. Populate(N , i , $nnarray$)
16. **endfor**

Métodos de muestreo

```

Populate( $N, i, nnarray$ ) (* Function to generate the synthetic samples. *)
17. while  $N \neq 0$ 
18.     Choose a random number between 1 and  $k$ , call it  $nn$ . This step chooses one of
        the  $k$  nearest neighbors of  $i$ .
19.     for  $attr \leftarrow 1$  to  $numattrs$ 
20.         Compute:  $dif = Sample[nnarray[nn]][attr] - Sample[i][attr]$ 
21.         Compute:  $gap =$  random number between 0 and 1
22.          $Synthetic[newindex][attr] = Sample[i][attr] + gap * dif$ 
23.     endfor
24.      $newindex++$ 
25.      $N = N - 1$ 
26. endwhile
27. return (* End of Populate. *)
    End of Pseudo-Code.

```

Consider a sample (6,4) and let (4,3) be its nearest neighbor.
 (6,4) is the sample for which k-nearest neighbors are being identified.
 (4,3) is one of its k-nearest neighbors.

Let:

$$f1_1 = 6 \quad f2_1 = 4 \quad f2_1 - f1_1 = -2$$

$$f1_2 = 4 \quad f2_2 = 3 \quad f2_2 - f1_2 = -1$$

The new samples will be generated as

$$(f1', f2') = (6,4) + \text{rand}(0-1) * (-2, -1)$$

$\text{rand}(0-1)$ generates a random number between 0 and 1.

Outline

- 1 **Preprocesamiento**
 - Introducción
 - Limpieza de Datos
 - Transformación de datos
 - Balanceo de clases
 - **Agrupamiento**
 - Introducción
 - Agrupamiento k-Means
 - Agrupamiento k-Medians
 - Agrupamiento k-Medoids
 - Extracción de características con PCA

Introduction

Now, we turn our attention to the unsupervised case, where class labeling of the training patterns is not available. Here, our major concern is to “reveal” the organization of patterns into “sensible” clusters (groups), which will allow us to discover similarities and differences among patterns and to derive useful conclusions about them.

Clustering may be found under different names in different contexts, such as unsupervised learning and learning without a teacher (in PR), numerical taxonomy (in biology, ecology), typology (in social sciences), and partition (in graph theory).

Example: Consider the following animals: sheep, dog, cat (mammals), sparrow, seagull (birds), viper, lizard (reptiles), goldfish, red mullet, blue shark (fish), and frog (amphibians).

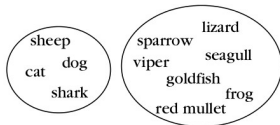
Introduction

In order to organize these animals into clusters, we need to define a clustering criterion.

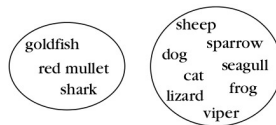
Following the next clustering criteria, we can form the clusters showed in the next figure. Criteria: a) the way these animals bear their progeny, b) the existence of lungs, c) the environment where the animals live, d) the way these animals bear their progeny **and** the existence of lungs.

This example shows that the process of assigning objects to clusters may lead to very different results, depending on the specific criterion used for clustering.

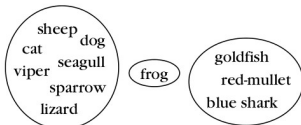
Introduction



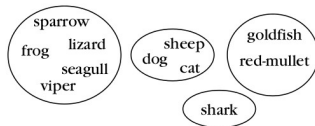
(a)



(b)



(c)



(d)

Introduction

In the clustering task, we will assume that all patterns are represented in terms of **features**, which form I -dimensional feature vectors.

The basic steps that an expert must follow in order to develop a clustering task are the following:

- **Feature selection.** Features must be properly selected so as to encode as much information as possible concerning the task of interest. Here, **preprocessing** of features may be necessary prior to their utilization in subsequent stages.
- **Proximity measure.** This measure quantifies how “similar” or “dissimilar” two feature vectors are.

Introduction

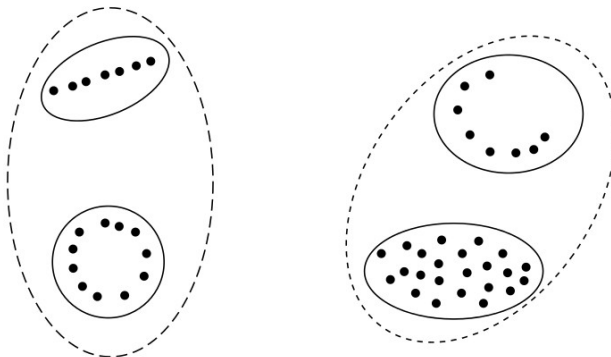
- **Clustering criterion.** This criterion depends on the interpretation the expert gives to the term *sensible*, based on the type of clusters that are expected to underlie the data set. The clustering criterion may be expressed via a cost function or some other types of rules.
- **Clustering algorithms.** Having adopted a proximity measure and a clustering criterion, this step refers to the choice of a specific algorithmic scheme that unravels the clustering structure of the data set.
- **Validation of the results.** Once the results of the clustering algorithm have been obtained, we have to verify their correctness. This is carried out using appropriate tests.

Introduction

- **Interpretation of the results.** In many cases, the expert in the application field must integrate the results of clustering with other experimental evidence and analysis in order to draw the right conclusions.

Different choices of features, proximity measures, clustering criteria, and clustering algorithms may lead to totally different clustering results. For example, consider the next figure. How many “sensible” ways of clustering can we obtain for these points? Which clustering is “correct”? Both clusterings are valid. The best thing to do is give the results to an expert and let the expert decide about the most sensible one.

Introduction



Applications of Cluster Analysis

- **Data reduction.** In several cases, the amount of the available data, N , is often very large and as a consequence, its processing becomes very demanding. Cluster analysis can be used in order to group the data into a number of “sensible” clusters, $m (\ll N)$, and to process each cluster as a single entity.
- **Hypothesis generation.** In this case we apply cluster analysis to a data set in order to infer some hypotheses concerning the nature of the data. These hypotheses must then be verified using other data sets.
- **Hypothesis testing.** Cluster analysis is used, here, for the verification of the validity of a specific hypothesis

Applications of Cluster Analysis

- **Prediction based on groups.** Here, cluster analysis is applied to the available data set, and the resulting clusters are characterized based on the characteristics of the patterns by which they are formed. In the sequel, if we are given an unknown pattern, we can determine the cluster to which it is more likely to belong, and we characterize it based on the characterization of the respective cluster.

Definition of Clustering

Many definitions of clustering have been proposed over the years. However, most of these definitions are of vague and of circular nature. This fact reveals the difficulty of having a universally acceptable definition for the term cluster.

For example, one definition of clustering, which is closer to our visual perception of clusters in the two- and three-dimensional spaces, is the following. The vectors are viewed as points in the I -dimensional space, and the clusters are described as continuous regions of this space containing a relatively high density of points, separated from other high density regions by regions of relatively low density of points.

From now on, we will follow the next definition. Let X be our data set, that is,

$$X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}.$$

Definition of Clustering

We define as an m -clustering of X , the partition of X into m sets (clusters), $C_1, \dots, C_m$, so that the following three conditions are met:

- $C_i \neq \emptyset, i = 1, \dots, m$
- $\bigcup_{i=1}^m C_i = X$
- $C_i \cap C_j = \emptyset, i \neq j, i, j = 1, \dots, m$

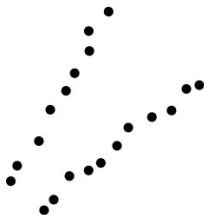
In addition, the vectors contained in a cluster C_i are “more similar” to each other and “less similar” to the feature vectors of the other clusters. Quantifying the terms similar and dissimilar depends very much on the types of clusters involved.

For example, other measures (measuring similarity) are required for compact clusters, others for elongated clusters, and different ones for shell-shaped clusters as showed in the next figure.

Definition of Clustering



(a)



(b)



(c)

Agrupamiento k -Means

Este agrupamiento es un procedimiento iterativo como sigue. Suponga que cuenta con una muestra de $\mathcal{X} = \{\mathbf{x}^t\}_{t=1}^N$ y se tienen k vectores referencia, $\mathbf{m}_j, j = 1, \dots, k$.

Primero, comenzamos con los $\mathbf{m}_j$ inicializados aleatoriamente. Después en cada iteración, usamos la siguiente ecuación y calculamos b_j^t para todos $\mathbf{x}^t$, los cuales son las etiquetas estimadas.

$$b_j^t = \begin{cases} 1 & \text{si } \|\mathbf{x}^t - \mathbf{m}_j\| = \min_j \|\mathbf{x}^t - \mathbf{m}_j\| \\ 0 & \text{en otro caso} \end{cases}$$

Si b_j^t es 1, se dice que $\mathbf{x}^t$ pertenece al grupo de $\mathbf{m}_j$. Una vez que obtenemos estas etiquetas, se minimiza la sig. ecuación.

Agrupamiento k -Means

$$E(\{\mathbf{m}_i\}_{i=1}^k | \mathcal{X}) = \sum_t \sum_i b_i^t \|\mathbf{x}^t - \mathbf{m}_i\|^2$$

Tomando su derivada con respecto a $\mathbf{m}_i$ y fijándola a 0, obtenemos

$$\mathbf{m}_i = \frac{\sum_t b_i^t \mathbf{x}^t}{\sum_t b_i^t}$$

Esto es, el vector referencia se fija a la media de todas las instancias que este representa.

Este es un procedimiento iterativo, ya que una vez que calculamos el nuevo $\mathbf{m}_i$, b_i^t cambia y necesita ser recalculado, el cual afectará después a $\mathbf{m}_i$. Estos dos pasos son repetidos hasta que $\mathbf{m}_i$ se estabiliza.

Agrupamiento k -Means

Initialize $\mathbf{m}_i, i = 1, \dots, k$, for example, to k random $\mathbf{x}^t$
Repeat
 For all $\mathbf{x}^t \in \mathcal{X}$

$$b_i^t \leftarrow \begin{cases} 1 & \text{if } \|\mathbf{x}^t - \mathbf{m}_i\| = \min_j \|\mathbf{x}^t - \mathbf{m}_j\| \\ 0 & \text{otherwise} \end{cases}$$

 For all $\mathbf{m}_i, i = 1, \dots, k$

$$\mathbf{m}_i \leftarrow \sum_t b_i^t \mathbf{x}^t / \sum_t b_i^t$$

Until $\mathbf{m}_i$ converge

Una desventaja del algoritmo es que es un procedimiento de búsqueda local, y el $\mathbf{m}_i$ final depende bastante del $\mathbf{m}_i$ inicial.

Agrupamiento k -Means

En resumen, el algoritmo k -means encuentra grupos en los datos, donde los grupos son representados por sus centros, los cuales se consideran los representativos típicos de su grupo.

Este agrupamiento también es utilizado para *preprocesamiento*, antes de una etapa posterior de clasificación o regresión.

Agrupamiento k -Medians

Este algoritmo es una variante de k -Means con la diferencia en que utiliza la distancia Manhattan (o métrica l_1) para minimizar el error.

Otra diferencia es que en la reasignación de los k centroides se calcula el vector cuyos elementos son iguales al valor **mediano** de cada dimensión de todas las instancias asignadas al cluster correspondiente.

En la práctica puede mejorar los resultados de k -means, principalmente para conjuntos que presentan datos atípicos (outliers).

Agrupamiento *k*-Medoids

Al igual que *k*-means, el agrupamiento *k*-medoids intenta minimizar la distancia entre puntos que se añadirían a un grupo y otro punto designado como el centro de ese grupo. La diferencia es que éste escoge elementos del conjunto como centros y trabaja con una métrica arbitraria de distancias entre datapoints en vez de usar la norma l_2 .

Se define un **medoid** como el objeto de un grupo cuya disimilaridad media a todos los objetos en el grupo es mínima. Es el punto ubicado más hacia el centro en todo el grupo.

Agrupamiento *k*-Medoids

Es decir, se minimiza la sig. ecuación

$$E(\{\mathbf{x}_i\}_{i=1}^k | \mathcal{X}) = \sum_{t \text{ noMed}} \sum_{i \text{ med}} b_i^t \|\mathbf{x}^t - \mathbf{x}^i\|^2$$

Agrupamiento k -Medoids

Algoritmo:

- ➊ **Inicialización:** seleccionar k de los n puntos como el medoid.
- ➋ **Asociar** cada punto al medoid más cercano.
- ➌ **Mientras** el costo de la configuración disminuya:
 - ➏ **Para** cada medoid m , para cada no medoid o :
 - ➐ Intercambiar m y o , recalcular el costo (suma de la distancia de los puntos a sus medoids).
 - ➑ Si el costo total de la configuración aumentó en el paso anterior, deshacer el intercambio.

Outline

- 1 Preprocesamiento
 - Introducción
 - Limpieza de Datos
 - Transformación de datos
 - Balanceo de clases
 - Agrupamiento
 - Introducción
 - Agrupamiento k-Means
 - Agrupamiento k-Medians
 - Agrupamiento k-Medoids
 - Extracción de características con PCA

Introduction

Principal Component Analysis (PCA) is used for dimensionality reduction, feature extraction, and data visualization mainly. It is also known as the Karhunen-Loève transform.

It can be defined in two ways:

- as the orthogonal projection of the data onto a lower dimensional linear space, known as the principal subspace, such that the variance of the projected data is maximized.
- as the linear projection that minimizes the average projection cost, defined as the mean squared distance between the data points and their projections.

Introduction

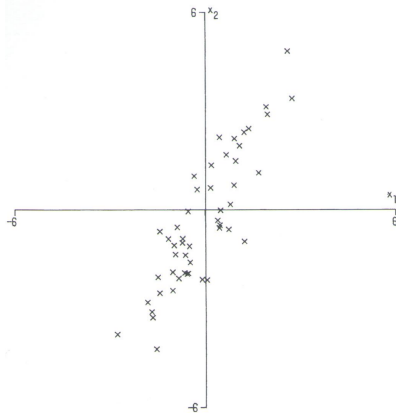
La idea central de PCA es reducir la dimensionalidad de un conjunto de datos (formado por un número grande de variables interrelacionadas), mientras retiene tanto como sea posible la variación presente en el conjunto de datos.

Lo anterior se logra mediante una transformación a un nuevo conjunto de variables, llamados Componentes Principales (PC) los cuáles no están correlacionados, y están ordenados $\vdash$ los primeros pocos retienen la mayoría de la variación presente en todas las variables originales.

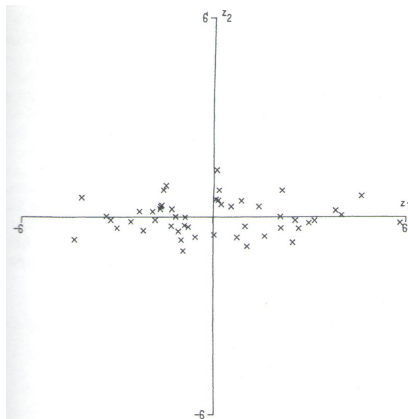
En las figuras siguientes se muestra un ejemplo.

Introduction

Datos



PCA



PCA: Maximum Variance Formulation

Consider a data set of observations $\{\mathbf{x}_n\}$ where $n = 1, \dots, N$, and $\mathbf{x}_n$ is a Euclidean variable with dimensionality D . The goal is to project the data onto a space having dimensionality $M < D$ while maximizing the variance of the projected data.

First, consider the projection onto a one-dimensional space. Define the direction of this space using a D -dimensional vector $\mathbf{u}_1$ (which is a unit vector so that $\mathbf{u}_1^T \mathbf{u}_1 = 1$). Each data point $\mathbf{x}_n$ is then projected onto a scalar value $\mathbf{u}_1^T \mathbf{x}_n$. The mean of the projected data is $\mathbf{u}_1^T \bar{\mathbf{x}}$ (where $\bar{\mathbf{x}} = 1/N \sum_{n=1}^N \mathbf{x}_n$).

PCA: Maximum Variance Formulation

The variance of the projected data is given by

$$\frac{1}{N} \sum_{n=1}^N \{(\mathbf{u}_1^T \mathbf{x}_n - \mathbf{u}_1^T \bar{\mathbf{x}})\}^2 = \mathbf{u}_1^T \mathbf{S} \mathbf{u}_1$$

where $\mathbf{S}$ is the data covariance matrix defined by

$$\mathbf{S} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^T$$

We now maximize the projected variance $\mathbf{u}_1^T \mathbf{S} \mathbf{u}_1$ with respect to $\mathbf{u}_1$. As a constrained maximization, the appropriate constraint comes from $\mathbf{u}_1^T \mathbf{u}_1 = 1$. A Lagrange multiplier λ_1 is introduced (to enforce this constraint), and then make an unconstrained maximization of

$$\mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 + \lambda_1 (1 - \mathbf{u}_1^T \mathbf{u}_1)$$

PCA: Maximum Variance Formulation

By setting the derivative with respect to $\mathbf{u}_1$ equal to zero $\Rightarrow$ this quantity will have a stationary point when $\mathbf{S}\mathbf{u}_1 = \lambda_1 \mathbf{u}_1$.

Therefore, $\mathbf{u}_1$ must be an eigenvector of $\mathbf{S}$. We see that the variance is given by (before, we left-multiply by $\mathbf{u}_1^T$ and make use of $\mathbf{u}_1^T \mathbf{u}_1 = 1$)

$$\mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 = \lambda_1$$

so the variance will be a maximum when we set $\mathbf{u}_1$ equal to the eigenvector having the largest eigenvalue λ_1 . This eigenvector is known as the **first principal component**.

Additional principal components can be defined in an incremental fashion by choosing each new direction to be that which maximizes the projected variance amongst all possible directions orthogonal to those already considered.

PCA: Maximum Variance Formulation

In summary, principal component analysis involves evaluating the mean $\bar{\mathbf{x}}$ and the covariance matrix $\mathbf{S}$ of the data set and then finding the M eigenvectors of $\mathbf{S}$ corresponding to the M largest eigenvalues.

Student's exercise: Download the Iris dataset from UCI repository and experiment with the PCA technique.

Properties and limitations of PCA

- Assumption on linearity. We assumed the observed data set to be linear combinations of certain basis.
- Assumption on the statistical importance of mean and covariance. PCA uses the eigenvectors of the covariance matrix and it only finds the independent axes of the data under the Gaussian assumption. However, there is no guarantee that the directions of maximum variance will contain good features for discrimination.

Properties and limitations of PCA

- Assumption that large variances have important dynamics. PCA simply performs a coordinate rotation that aligns the transformed axes with the directions of maximum variance. That is, we believe that the observed data has a high signal-to-noise ratio that the principal components with larger variance correspond to interesting dynamics and lower ones correspond to noise.

Applications of PCA

- Data compression. PCA allows an approximation for each data vector $\mathbf{x}_n$.
- Data pre-processing. Standardizing the data such that each variable had zero mean and unit variance.
- Data visualization.